

School LLM — Project Status Report

Prepared by: Gangadhar Reddy

Date: 26 May 2026

Project: School LLM — AI-Powered Learning Platform

Status: Active Development — Production Hardening Phase

1. Executive Summary

School LLM is a multi-role, AI-assisted learning platform designed for K-12 institutions. The system enables teachers to publish course materials, administer assignments, and review submissions, while empowering students to study those materials through AI-generated summaries, quizzes, narrated audio explanations, video walkthroughs, and an interactive Q&A chat that answers questions strictly from the assigned source material. Administrators retain full oversight of users, roles, content, analytics, and the AI evaluation pipeline.

The project has progressed from prototype to a feature-complete application that is now entering its production-hardening phase. The current sprint focuses on operational controls — specifically administrator-managed rate limiting — and the next sprint will be dedicated to end-to-end testing and quality assurance.

2. Technology Stack

- Backend API built on FastAPI (Python 3.11).
- Frontend implemented as a multi-page Streamlit application.
- Primary datastore is MongoDB Atlas.
- Vector store provided by ChromaDB, persisted to local storage.
- Local language model served by Ollama, running qwen2.5:3b.
- Cloud language model integration with Anthropic Claude (Sonnet and Haiku tiers) for fallback and evaluation.
- Embeddings generated using the Sentence-Transformers model all-MiniLM-L6-v2.
- Authentication implemented with JWT tokens (HS256) and bcrypt-hashed credentials.
- PDF processing handled by PyPDF and a custom semantic chunker.
- Audio generation provided by a local text-to-speech engine.
- Video generation implemented with a MoviePy-based slide-to-video pipeline.

3. Completed Functionality

3.1 Authentication and Role Management

- Email and password authentication with bcrypt-hashed credentials.
- JWT-based session tokens, persisted in the Streamlit URL query parameters for refresh-safe sessions.
- Three-tier role hierarchy: Administrator, Teacher, and Student.

- A dedicated role-permissions matrix that allows administrators to grant or revoke individual feature access per role without requiring redeployment.
- Per-route authorisation dependencies consistently enforced across the FastAPI surface.

3.2 Student Module

- A PDF Library where students view the documents assigned to their class and section, and select one as the active study material.
- An AI Q&A Chat that answers student questions strictly from the active PDF, grounded in retrieval-augmented generation against the ChromaDB vector store.
- Chat history persistence on a per-session basis, with new sessions auto-initialising as a fresh chat.
- A Summary Generator that produces concise, structured summaries adapted to the source document type and consistently ending in complete sentences.
- A Quiz Generator that creates mixed-format assessments including multiple-choice, true/false, fill-in-the-blank, short-answer, and long-answer questions. Objective questions are auto-scored, and free-text responses are AI-judged.
- An Audio Explanation feature that produces a narrated audio walkthrough of the selected material, stored to the runtime data folder for repeated playback.
- A Video Walkthrough feature that produces a slide-based video explanation, stored alongside the audio assets.
- Full artifact persistence so that all generated content, including quizzes, summaries, audio, video, and chats, remains visible to the student across logouts.

3.3 Teacher Module

- Class and subject scoping so that teachers see only the classes and subjects to which they have been assigned.
- An assignment creation workflow supporting both manual authoring and AI-generated question sets sourced from an attached PDF.
- Exposure of the same AI generation pipeline used for student quizzes, enabling teachers to author assignments with AI assistance.
- A submission review interface that displays student submissions, applies AI-suggested grades, and supports manual score overrides with comments.
- Full activity tracking, with every teacher action recorded in the centralised audit log used by administrators.

3.4 Administrator Module

The Administrator Dashboard is organised into the following functional areas:

- Analytics: aggregated usage metrics across students, teachers, content, and AI features.
- Users: user lifecycle management, including activation, deactivation, class assignment, and teacher-to-subject mapping.

- Roles and Permissions: a live matrix that toggles feature access per role without redeployment.
- Activity: a time-ordered audit log of all user-initiated actions.
- All PDFs: a repository-wide view of uploaded documents with full metadata.
- AI Evaluation: a reference-free, LLM-as-judge evaluation system providing two one-click bundles (Student Evaluation and Teacher Evaluation) that score real production traffic on faithfulness, completeness, and correctness.
- Export: audit log and analytics export for compliance and reporting.
- Rate Limits: per-day quotas for AI features, configurable per role. This is the focus of the current sprint and is described in Section 4.

3.5 AI Pipeline and Resilience

- Retrieval-augmented generation across Q&A, summarisation, and quiz workflows. PDF documents are chunked, embedded, and indexed in ChromaDB, and the top-k relevant chunks are retrieved before any language-model call.
- A multi-provider language model abstraction with a unified client interface that routes generation, evaluation, and naming calls to the configured backend, switchable via environment variables.
- Graceful degradation through a fallback client wrapper that automatically retries failed requests on the alternate provider, with a configurable cooldown so a temporarily unavailable provider is not repeatedly hammered.
- Prompt caching for the Anthropic provider, where static system prompts are sent with ephemeral cache markers, reducing latency and cost for batched evaluations.
- LLM-as-judge evaluation that self-assesses output quality across three independent metrics: semantic match, completeness, and faithfulness. Prompts are calibrated to award partial credit in the 0.4 to 0.9 band rather than binary 0 or 1 scores, and the judge receives up to fifteen thousand characters of source context for reliable verification.

4. Current Sprint — Administrator-Controlled Rate Limiting

The platform is being prepared for production rollout, where uncontrolled AI usage could exhaust compute resources or incur unnecessary cost. The current sprint introduces administrator-controlled daily quotas for every AI feature.

4.1 Design

- A singleton settings document in MongoDB stores per-role, per-feature limits. A separate counter collection tracks each user's daily consumption, keyed by user identifier, feature, and calendar day.
- A reusable FastAPI dependency, applied to each AI endpoint, atomically increments the user's daily counter on every request and returns HTTP 429 Too Many Requests when the configured limit is exceeded, accompanied by the number of seconds remaining until midnight reset.
- A new Rate Limits tab in the Administrator Dashboard presents two side-by-side sections — one for the Student module and one for the Teacher module — each listing the five AI features with editable numeric inputs. A value of negative one denotes unlimited usage, and a value of zero

disables the feature for that role. Changes apply on the next API call without requiring a backend restart.

- Administrators are explicitly exempt from rate limiting at the server level, ensuring that configuration mistakes can never lock administrators out of their own platform.
- The per-day quota layer complements an existing per-minute anti-burst middleware (sixty requests per minute generally, twenty requests per minute on AI endpoints), which continues to operate as a denial-of-service safeguard.

4.2 Status

- The backend rate-limit module has been implemented.
- The rate-limit dependency has been wired into all AI endpoints, including Q&A, Summary, Quiz, Audio, and Video.
- The administrator REST endpoints for reading, updating, and inspecting per-user usage have been implemented.
- The Administrator Dashboard Rate Limits tab has been implemented with dual Student and Teacher sections.
- OpenAPI verification of the registered routes has passed.
- Authentication-gating verification on protected routes has passed.
- The end-to-end live test, in which an administrator sets a quota, a student exceeds it, and the system responds with HTTP 429, is pending and scheduled for completion in the current sprint.

5. Future Scope — Testing and Quality Assurance

The next sprint will focus on a comprehensive testing programme to certify the platform for production deployment.

5.1 Planned Test Coverage

- Unit testing across pure-Python modules, including PDF chunking, prompt construction, rate-limit window arithmetic, JSON judge-response parsing, and authorisation helpers.
- Integration testing of FastAPI endpoint contracts against an ephemeral MongoDB instance, covering authentication, role-based authorisation, rate limiting, AI orchestration, and administrator workflows.
- End-to-end testing of Streamlit user journeys for each of the three roles, automated using Playwright. The journeys covered are the student study workflow, the teacher assignment workflow, and the administrator management workflow.
- Load testing through sustained-traffic simulations against AI endpoints, validating the rate-limit ceiling, confirming graceful HTTP 429 behaviour, and measuring latency under concurrent load.
- Security review, including authentication and authorisation hardening, JWT expiry handling, brute-force protection, input validation on file uploads, and dependency vulnerability scanning.

- AI quality regression, achieved by scheduling regular execution of the in-product Student and Teacher Evaluation bundles to detect regressions in answer faithfulness and completeness over time.

5.2 Acceptance Criteria for Production Readiness

- All unit and integration tests passing in continuous integration.
- End-to-end suites passing on the three primary user journeys.
- Rate-limit enforcement validated under simulated load.
- AI evaluation scores stable across two consecutive evaluation cycles.
- Security review checklist fully cleared.

6. Conclusion

School LLM has reached feature completeness across the three role-based modules and the AI generation pipeline. The platform's core differentiators — multi-provider language-model resilience, self-evaluating output quality, and a role-aware administrative surface — are in place and operating against production data.

The current rate-limiting sprint addresses the final operational gap before launch, and the subsequent testing sprint will provide the verification required to certify the application for institutional deployment. The project remains on track for production readiness within the next development cycle.

End of report.